Crypto – TME – Notes

FLAG 6 : Générateur de clé RSA

Objectif :

- Générer une paire de clés RSA dont l'exposant public *e* est fabriqué à partir de la chaîne « +++ATRIUM+++ »

Méthode générale :

- Fabriquer un exposant RSA personnalisé à partir de « +++ATRIUM+++ ».
- Générer une clé privée RSA avec cet exposant via OpenSSL.
- En extraire la clé publique.
- Vérifier si la chaîne « +++ATRIUM+++ » apparaît dans la version Base64 de cette clé publique.

Explication du code :

- Imports (L1-L3)
- Définition de la cible en base64 (L5)
- Dossiers input/output (L8-L9)
- Décodage de la cible de Base64 vers des octets binaires (L12)
- Forcer un entier impair en ajoutant l'octet b"\x01" (L15)
- Convertir les octets en entier pour en faire l'exposant RSA et y ajouter un très grand nombre pour gonfler la valeur finale (L14)

```
atrium_int = int.from_bytes(atrium_b64_dec, byteorder="big") + (256 * 10**100)
```

- Construction de la commande OpenSSL pour générer la clé privée (L23-31)

```
print("Génération de la clé privée avec OpenSSL...")
cmd_keygen = [
    "openssl", "genpkey", # Lance OpenSSL et la sous-commande de génération de clé privée.
    "-algorithm", "RSA", # Demande une clé RSA.
    "-out", str(sk_path), # Fichier de sortie de la clé privée.
    "-outform", "PEM", # Format PEM:fichier texte encadré par des lignes "-----BEGIN ...-----".
    "-pkeyopt", f"rsa_keygen_pubexp:{atrium_int}", # Définit l'exposant public RSA e à la valeur cible.
    "-pkeyopt", "rsa_keygen_bits:2048", # Taille de la clé RSA : 2048 bits.
    "-pkeyopt", "rsa_keygen_primes:2", # Demande une RSA classique à deux nombres premiers p et q.
]
```

- Exécution de la commande OpenSSL (L33)
- Affichage du code de retour du processus, des erreurs, et de la sortie normale (L34-L36)
- Test de vérification du bon lancement de la commande OpenSSL (L38-L39)
- Extraction de la clé publique depuis la clé privée (L42-L47)

```
cmd_pubout = [
    "openssl", "pkey", # Manipule une clé déjà existante.
    "-in", str(sk_path), # Lit la clé privée générée juste avant.
    "-pubout", # Extrait seulement la partie publique.
    "-out", str(pk_path), # Ecrit la clé publique dans un autre fichier.
]
```

- Exécution de la commande OpenSSL (L49)
- Affichage du code de retour du processus, des erreurs, et de la sortie normale (L50-L52)
- Test de vérification du bon lancement de la commande OpenSSL (L54-L55)
- Lit tout le fichier PEM public comme texte (L62)
- Découpe le texte ligne par ligne (L65)
- Reconstruit une seule grande chaîne en concaténant toutes les lignes qui ne contiennent pas ----- (L66)
- Teste si la version texte de TARGET, donc "+++ATRIUM+++", est présente comme sous-chaîne dans la Base64 de la clé publique (L68)

Affiche l’échec si la chaîne n’est pas retrouvée.

FLAG 13 : Shared Modulus (RSA)

Objectif :

- Retrouver le Plaintext message m qui a été chiffré par deux individus avec la méthode RSA utilisant le même modulus N.

Rappel (construction de la clé privée en RSA) :

- Generate two large primes p and q with $\mathrm{p} \neq \mathrm{q}$.
- Compute:
$$\mathrm{N} = \mathrm{p} \times \mathrm{q}$$
- Compute Euler's totient:
$$\varphi(\mathrm{N}) = (\mathrm{p} - 1) \times (\mathrm{q} - 1)$$
- Choose a random public exponent e (common choice is $\mathrm{e} = 65537$) such that:
$$\begin{cases} 1 < \mathrm{e} < \varphi(\mathrm{N}) \\ \gcd(\mathrm{e}, \varphi(\mathrm{N})) = 1 \end{cases}$$
- Compute the unique integer d such that:
$$\begin{cases} 1 < \mathrm{d} < \varphi(\mathrm{N}) \\ \mathrm{d} \times \mathrm{e} \equiv 1 \bmod \varphi(\mathrm{N}) \end{cases}$$
- **The public key is (N, e).**
- **The private key is (d, p, q).**

Contexte :

- Je connais les clés publiques (e_1, N_1) et (e_2, N_2).
- Le module est le même : $N_1 = N_2 = N$.
- Je connais les chiffrés du même message c_1 et c_2.
- Le message m est tel que $0 \leq m < N$.

Attaque :

- Vérifier que e_1 et e_2 sont premier entre eux : $\gcd(e_1, e_2) = 1$.
- D'après le théorème de Bézout, comme e_1 et e_2 sont premier entre eux :
$$u \cdot e_1 + v \cdot e_2 = 1$$
- Mettre le chiffré c_1à la puissance u et le chiffré c_2à la puissance v :
$$\begin{cases} c_1^u = m^{e_1 u} \; mod \; N = m^{e_1 \cdot u} \; mod \; N \\ c_2^v = m^{e_2 v} \; mod \; N = m^{e_2 \cdot v} \; mod \; N \end{cases}$$
- Attention, comme soit a soit b est négatif (supposons $a < 0$), je ne peux pas calculer c_1^a directement, donc je dois passer par l'inverse modulaire :
$$c_1^{-|a|} = (c_1^{-1})^{|a|} \; mod \; N$$
- Et l'inverse modulaire de c_1 existe car $gcd(c_1, N) = 1$, sinon on aurait
- factorisé N, ce qui serait encore mieux.
- Je multiplie ces 2 chiffrés c_1^u et c_2^v entre eux :
$$c_1^u \cdot c_2^v = m^{e_1 \cdot u} \cdot m^{e_2 \cdot v} \; mod \; N = m^{e_1 \cdot u + e_2 \cdot v} \; mod \; N = m \; mod \; N = m$$
- J'ai retrouvé le message m !

Remarque :

- Quand $e_1$et e_2 ne sont pas premiers entre eux, je pose :

$$d = \gcd(e_1, e_2) > 1$$

- Tel que :

$$\begin{cases} e_1 = d \cdot a \\ e_2 = d \cdot b \\ \gcd\,(a, b) = 1 \end{cases}$$

- Comme a et b sont premiers entre eux, il existe (u, v) tels que :

$$u \cdot a + v \cdot b = 1$$

- Donc :

$$u \cdot e_1 + v \cdot e_2 = d$$

- À partir des chiffrés, on calcule :
  $X = c_1^u \cdot c_2^v \bmod N = m^{e_1{}^u} \cdot m^{e_2{}^v} \bmod N = m^{e_1 \cdot u + e_2 \cdot v}\ mod\ N = m^d\ mod\ N = m^d$
- Il suffit de prendre la racine d-ième entière :

$$m = \sqrt[d]{X}$$

- J'ai retrouvé le message m !

FLAG 15 : Attaque de Haastard (petit exposant publique commun)

Objectif :

- Retrouver le Plaintext message m qui a été chiffré par trois individus avec la méthode RSA utilisant le même petit exposant e (passer par le théorèe chinois des restes).

Contexte :

- Je connais les chiffrés de 3 individus :

$$\begin{cases} c_1 \equiv m^e \ mod \ N_1 \\ c_2 \equiv m^e \ mod \ N_2 \\ c_3 \equiv m^e \ mod \ N_3 \end{cases}$$

- Je connais les clés publiques (e_1, N_1), (e_2, N_2), et (e_3, N_3).
- Le message m est tel que $0 \leq m < N$.

Attaque :

- Mathématiquement, je cherche un entier X tel que :

$$\begin{cases} X \equiv c_1 \bmod N_1 \\ X \equiv c_2 \bmod N_2 \\ X \equiv c_3 \bmod N_3 \end{cases}$$

- Comme :

$$c_i \equiv m^e \bmod N_i$$

- Cela revient à chercher un X vérifiant :

$$\begin{cases} X \equiv m^e \bmod N_1 \\ X \equiv m^e \bmod N_2 \\ X \equiv m^e \bmod N_3 \end{cases}$$

- Le théorème chinois des restes dit que si N_1, N_2 et N_3 sont deux à deux premiers entre eux, alors ce système admet une solution unique modulo $M = N_1 N_2 N_3$.
- Autrement dit, on peut reconstruire un entier X qui satisfait les trois congruences tel que :

$$0 \leq X < M$$

- Donc :

$$X \equiv m^e \bmod N_1 N_2 N_3 \equiv m^e \ mod \ M$$

- Pour calculer ce X, on pose :

$$\begin{cases} M = N_1 N_2 N_3 \\ M_i = \dfrac{M}{N_i} \\ u_i \equiv M_i^{-1} \bmod N_i \end{cases}$$

- D'où la formule explicite pour calculer X :

$$X \equiv c_1 M_1 u_1 + c_2 M_2 u_2 + c_3 M_3 u_3 \bmod M = m^e \ mod \ M = m^e$$

- Je peux alors calculer :

$$m = \sqrt[e]{X}$$

- J'ai retrouvé le message m !

<u>Preuve de la validité du théorème chinois des restes :</u>

- A partir du système de congruences suivant :

$$\begin{cases} X \equiv m^e \bmod N_1 \\ X \equiv m^e \bmod N_2 \\ X \equiv m^e \bmod N_3 \end{cases} \Rightarrow \begin{cases} (X - m^e) \equiv N_1 \\ (X - m^e) \equiv N_2 \\ (X - m^e) \equiv N_3 \end{cases}$$

- Posons :

$$z = X - m^e$$

- On sait :

$$\begin{cases} N_1 \mid z \\ N_2 \mid z \\ N_3 \mid z \end{cases}$$

- Je veux montrer que :

$$N_1 N_2 N_3 \mid z$$

- **Étape 1 : combiner N_1et N_2**
- Comme $N_1 \mid z$, il existe $a \in \mathbb{Z}$ tel que :

$$z = N_1 a$$

- Mais comme aussi $N_2 \mid z$, donc :

$$N_2 \mid N_1 a \Rightarrow N_2 | N_1 \; ou \; N_2 | a$$

- Or :

$$\gcd\,(N_1, N_2) = 1$$

- Donc N_2 ne divise pas N_1, donc :

$$N_2 \mid a$$

- Donc il existe $b \in \mathbb{Z}$ tel que :

$$a = N_2 b$$

- En remplaçant dans $z = N_1 a$, j'obtiens :

$$z = N_1 N_2 b$$

- Donc :

$$N_1 N_2 \mid z$$

- **Étape 2 : ajouter N_3**
- J'ai maintenant :

$$\begin{cases} N_1 N_2 \mid z \\ N_3 \mid z \end{cases}$$

- Je refais exactement le même raisonnement, comme $N_3 \mid z$, il existe $c \in \mathbb{Z}$ tel que :

$$z = N_3 c$$

- Mais comme aussi $N_1 N_2 \mid z$, donc :

$$N_1 N_2 \mid N_3 c \Rightarrow N_1 N_2 | N_3 \; ou \; N_1 N_2 | c$$

- Or N_1, N_2, N_3sont deux à deux premiers entre eux, donc $N_1 N_2$ ne divise pas N_3, alors :

$$N_1 N_2 \mid c$$

- Donc il existe $d \in \mathbb{Z}$ tel que :

$$c = N_1 N_2 d$$

- En remplaçant dans $z = N_3 c$, j'obtiens :

$$z = N_1 N_2 N_3 d$$

- Donc :

$$N_1 N_2 N_3 \mid z$$

- Donc :

$$N_1 N_2 N_3 \mid (X - m^e)$$

- Autrement dit :

$$X \equiv m^e \bmod N_1 N_2 N_3$$